

在 Node.js 運用大數據生成 Google 簡報

來源：<https://codelabs.developers.google.com/codelabs/slides-api?hl=zh-tw>

步驟數：9

在本程式碼研究室中，您將使用 Google Slides API 和 BigQuery 製作簡報，分析最常見的軟體授權分析。

目錄

1. [1. 總覽](#)
2. [2. 取得範例程式碼](#)
3. [3. 執行範例應用程式](#)
4. [4. 取得用戶端密鑰](#)
5. [5. 建立 OAuth2 用戶端](#)
6. [6. 設定 BigQuery](#)
7. [7. 建立簡報](#)
8. [8. 開啟簡報](#)
9. [9. 恭喜！](#)

1. 總覽

在本程式碼研究室中，您將瞭解如何使用 Google 簡報做為自訂簡報工具，分析最常見的軟體授權。您將使用 [BigQuery API](#) 查詢 [GitHub 上的所有開放原始碼](#)，並使用 [Google 簡報 API](#) 建立投影片組來呈現結果。範例應用程式是使用 Node.js 建構而成，但相同的基本原則適用於任何架構。

課程內容

- 使用 Google 簡報 API 建立簡報
- 使用 BigQuery 深入瞭解大型資料集
- 使用 Google Drive API 複製檔案

軟硬體需求

- 已安裝 Node.js
 - 連上網際網路並使用網路瀏覽器
 - Google 帳戶
 - Google Cloud Platform 專案
-

步驟 2 · 約 1 分鐘

2. 取得範例程式碼

您可以將所有程式碼範例下載至電腦...

...或從指令列複製 GitHub 存放區。

```
git clone https://github.com/googleworkspace/slides-api.git
```

存放區包含一組目錄，代表程序中的每個步驟，方便您參照工作版本。

您將使用 `start` 目錄中的副本，但可以視需要參照或複製其他檔案。

3. 執行範例應用程式

首先，請啟動並執行 Node 指令碼。下載程式碼後，請按照下列操作說明安裝並啟動 Node.js 應用程式：

1. 在電腦上開啟指令列終端機，然後前往程式碼研究室的 `start` 目錄。
2. 輸入下列指令，安裝 Node.js 依附元件。

〇〇〇

```
npm install
```

3. 輸入下列指令來執行指令碼：

〇〇〇

```
node .
```

4. 請注意，畫面會顯示這個專案的步驟。

〇〇〇

```
-- Start generating slides. --  
TODO: Get Client Secrets  
TODO: Authorize  
TODO: Get Data from BigQuery  
TODO: Create Slides  
TODO: Open Slides  
-- Finished generating slides. --
```

您可以在 `slides.js`、`license.js` 和 `auth.js` 中查看我們的待辦事項清單。請注意，我們使用 [JavaScript Promise](#) 串連完成應用程式所需的步驟，因為每個步驟都必須完成前一個步驟才能進行。

如果您不熟悉 Promise，別擔心，我們會提供您所需的所有程式碼。簡而言之，Promise 可讓我們以更同步的方式處理非同步處理作業。

步驟 4 · 約 5 分鐘

4. 取得用戶端密鑰

如要使用 Slides、BigQuery 和 Drive API，我們將建立 OAuth 用戶端和服務帳戶。

設定 Google Developers Console

1. 使用[這個精靈](#)在 Google Developers Console 中建立或選取專案，並自動啟用 API。依序點選「繼續」和「前往憑證」。
2. 在「將憑證新增至專案」頁面中，按一下「取消」按鈕。
3. 選取頁面頂端的「OAuth 同意畫面」分頁標籤。選取「電子郵件地址」，輸入產品名稱 `Slides API Codelab`，然後按一下「儲存」按鈕。

啟用 BigQuery、Google 雲端硬碟和 Google 簡報 API

1. 選取「資訊主頁」分頁標籤，按一下「啟用 API」按鈕，然後啟用下列 3 項 API：
2. **BigQuery API**
3. **Google Drive API**
4. **Google Slides API**

下載 OAuth 用戶端密鑰 (適用於 Google 簡報和 Google 雲端硬碟)

1. 選取「憑證」分頁，按一下「建立憑證」按鈕，然後選取「OAuth 用戶端 ID」。
2. 選取「Other」(其他) 應用程式類型，輸入名稱 `Google Slides API Codelab`，然後按一下「Create」(建立) 按鈕。按一下「OK」(確定) 關閉隨即顯示的對話方塊。
3. 按一下用戶端 ID 右側的 `file_download` (下載 JSON) 按鈕。
4. 將密鑰檔案重新命名為 `client_secret.json`，然後複製到 **start/** 和 **finish/** 目錄。

下載服務帳戶密鑰 (適用於 BigQuery)

1. 選取「憑證」分頁標籤，按一下「建立憑證」按鈕，然後選取「服務帳戶金鑰」。
2. 在下拉式選單中，選取「New Service Account」(新增服務帳戶)。為服務選擇名稱 `Slides API Codelab service`。然後按一下「角色」，捲動至「BigQuery」，並選取「BigQuery 資料檢視者」和「BigQuery 工作使用者」。
3. 在「金鑰類型」中，選取「JSON」。
4. 按一下「建立」，金鑰檔案會自動下載到電腦。按一下「關閉」，即可結束顯示的對話方塊。
5. 將密鑰檔案重新命名為 `service_account_secret.json`，然後複製到 **start/** 和 **finish/** 目錄。

取得用戶端密鑰

在 `start/auth.js` 中，填入 `getClientSecrets` 方法。

[auth.js](#)

```
const fs = require('fs');

/**
 * Loads client secrets from a local file.
 * @return {Promise} A promise to return the secrets.
 */
module.exports.getClientSecrets = () => {
  return new Promise((resolve, reject) => {
    fs.readFile('client_secret.json', (err, content) => {
      if (err) return reject('Error loading client secret file: ' + err);
      console.log('loaded secrets...');
      resolve(JSON.parse(content));
    });
  });
}
```

我們現在已載入用戶端密鑰。憑證會傳遞至下一個 Promise。使用 `node .` 執行專案，確保沒有錯誤。

步驟 5 · 約 8 分鐘

5. 建立 OAuth2 用戶端

如要建立投影片，請將下列程式碼新增至 `auth.js` 檔案，為 Google API 新增驗證。這項驗證會要求存取您的 Google 帳戶，以便讀取及寫入 Google 雲端硬碟中的檔案、在 Google 簡報中建立簡報，以及從 Google BigQuery 執行唯讀查詢。
(注意：我們沒有變更 `getClientSecrets`)

`auth.js`


```

const fs = require('fs');
const readline = require('readline');
const openurl = require('openurl');
const googleAuth = require('google-auth-library');
const TOKEN_DIR = (process.env.HOME || process.env.HOMEPATH ||
  process.env.USERPROFILE) + '/.credentials/';
const TOKEN_PATH = TOKEN_DIR + 'slides.googleapis.com-nodejs-quickstart.json';

// If modifying these scopes, delete your previously saved credentials
// at ~/.credentials/slides.googleapis.com-nodejs-quickstart.json
const SCOPES = [
  'https://www.googleapis.com/auth/presentations', // needed to create slides
  'https://www.googleapis.com/auth/drive', // read and write files
  'https://www.googleapis.com/auth/bigquery.readonly' // needed for bigquery
];

/**
 * Loads client secrets from a local file.
 * @return {Promise} A promise to return the secrets.
 */
module.exports.getClientSecrets = () => {
  return new Promise((resolve, reject) => {
    fs.readFile('client_secret.json', (err, content) => {
      if (err) return reject('Error loading client secret file: ' + err);
      console.log('loaded secrets...');
      resolve(JSON.parse(content));
    });
  });
}

/**
 * Create an OAuth2 client promise with the given credentials.
 * @param {Object} credentials The authorization client credentials.
 * @param {function} callback The callback for the authorized client.
 * @return {Promise} A promise to return the OAuth client.
 */
module.exports.authorize = (credentials) => {
  return new Promise((resolve, reject) => {
    console.log('authorizing...');
    const clientSecret = credentials.installed.client_secret;
    const clientId = credentials.installed.client_id;
    const redirectUrl = credentials.installed.redirect_uris[0];
    const auth = new googleAuth();
    const oauth2Client = new auth.OAuth2(clientId, clientSecret, redirectUrl);

    // Check if we have previously stored a token.
    fs.readFile(TOKEN_PATH, (err, token) => {
      if (err) {
        getNewToken(oauth2Client).then(() => {
          resolve(oauth2Client);
        });
      } else {

```

```

        oauth2Client.credentials = JSON.parse(token);
        resolve(oauth2Client);
    }
    });
});
}

/**
 * Get and store new token after prompting for user authorization, and then
 * fulfills the promise. Modifies the `oauth2Client` object.
 * @param {google.auth.OAuth2} oauth2Client The OAuth2 client to get token for.
 * @return {Promise} A promise to modify the oauth2Client credentials.
 */
function getNewToken(oauth2Client) {
    console.log('getting new auth token...');
    openurl.open(oauth2Client.generateAuthUrl({
        access_type: 'offline',
        scope: SCOPES
    }));

    console.log(''); // \n
    return new Promise((resolve, reject) => {
        const rl = readline.createInterface({
            input: process.stdin,
            output: process.stdout
        });
        rl.question('Enter the code from that page here: ', (code) => {
            rl.close();
            oauth2Client.getToken(code, (err, token) => {
                if (err) return reject(err);
                oauth2Client.credentials = token;
                let storeTokenErr = storeToken(token);
                if (storeTokenErr) return reject(storeTokenErr);
                resolve();
            });
        });
    });
}

/**
 * Store token to disk be used in later program executions.
 * @param {Object} token The token to store to disk.
 * @return {Error?} Returns an error or undefined if there is no error.
 */
function storeToken(token) {
    try {
        fs.mkdirSync(TOKEN_DIR);
        fs.writeFileSync(TOKEN_PATH, JSON.stringify(token));
    } catch (err) {
        if (err.code !== 'EEXIST') return err;
    }
}

```

```
console.log('Token stored to ' + TOKEN_PATH);  
}
```

步驟 6 · 約 5 分鐘

6. 設定 BigQuery

探索 BigQuery (選用)

BigQuery 可讓我們在幾秒內查詢龐大的資料集。在以程式輔助方式查詢之前，請先使用網頁介面。如果您從未設定 BigQuery，請[按照這篇快速入門導覽課程中的步驟操作](#)。

開啟 [Cloud 控制台](#)，瀏覽 BigQuery 中提供的 GitHub 資料，並執行自己的查詢。請編寫這項查詢並按下「執行」按鈕，找出 GitHub 上最熱門的軟體授權。

bigquery.sql

```
WITH AllLicenses AS (  
  SELECT * FROM `bigquery-public-data.github_repos.licenses`  
)  
SELECT  
  license,  
  COUNT(*) AS count,  
  ROUND((COUNT(*) / (SELECT COUNT(*) FROM AllLicenses)) * 100, 2) AS percent  
FROM `bigquery-public-data.github_repos.licenses`  
GROUP BY license  
ORDER BY count DESC  
LIMIT 10
```

我們剛分析了 GitHub 上數百萬個公開存放區，找出最熱門的授權。分析成效非常出色！現在，我們來設定以程式輔助方式執行相同的查詢。

設定 BigQuery

將 `license.js` 檔案中的程式碼替換成以下程式碼。函式 `bigquery.query` 會傳回 [promise](#)。

license**.js**

```

const google = require('googleapis');
const read = require('read-file');
const BigQuery = require('@google-cloud/bigquery');
const bigquery = BigQuery({
  credentials: require('./service_account_secret.json')
});

// See codelab for other queries.
const query = `
WITH AllLicenses AS (
  SELECT * FROM \`bigquery-public-data.github_repos.licenses\`
)
SELECT
  license,
  COUNT(*) AS count,
  ROUND((COUNT(*) / (SELECT COUNT(*) FROM AllLicenses)) * 100, 2) AS percent
FROM \`bigquery-public-data.github_repos.licenses\`
GROUP BY license
ORDER BY count DESC
LIMIT 10
`;

/**
 * Get the license data from BigQuery and our license data.
 * @return {Promise} A promise to return an object of licenses keyed by name.
 */
module.exports.getLicenseData = (auth) => {
  console.log('querying BigQuery...');
  return bigquery.query({
    query,
    useLegacySql: false,
    useQueryCache: true,
  }).then(bqData => Promise.all(bqData[0].map(getLicenseText)))
    .then(licenseData => new Promise((resolve, reject) => {
      resolve([auth, licenseData]);
    }))
    .catch((err) => console.error('BigQuery error:', err));
}

/**
 * Gets a promise to get the license text about a license
 * @param {object} licenseDatum An object with the license's
 *   `license`, `count`, and `percent`
 * @return {Promise} A promise to return license data with license text.
 */
function getLicenseText(licenseDatum) {
  const licenseName = licenseDatum.license;
  return new Promise((resolve, reject) => {
    read(`licenses/${licenseName}.txt`, 'utf8', (err, buffer) => {
      if (err) return reject(err);
      resolve({
        licenseName,

```

```
        count: licenseDatum.count,  
        percent: licenseDatum.percent,  
        license: buffer.substring(0, 1200) // first 1200 characters  
    });  
  });  
});  
}
```

請嘗試 `console.log` Promise 回呼中的部分資料，瞭解物件結構並查看實際運作的程式碼。

步驟 7 · 約 7 分鐘

7. 建立簡報

現在來到有趣的部分！讓我們呼叫 Slides API 的 `create` 和 `batchUpdate` 方法來建立投影片。將檔案內容替換為下列程式碼：

slides.js

```
const google = require('googleapis');
const slides = google.slides('v1');
const drive = google.drive('v3');
const openurl = require('openurl');
const commaNumber = require('comma-number');

const SLIDE_TITLE_TEXT = 'Open Source Licenses Analysis';

/**
 * Get a single slide json request
 * @param {object} licenseData data about the license
 * @param {object} index the slide index
 * @return {object} The json for the Slides API
 * @example licenseData: {
 *     "licenseName": "mit",
 *     "percent": "12.5",
 *     "count": "1667029"
 *     license:"<body>"
 * }
 * @example index: 3
 */
function createSlideJSON(licenseData, index) {
    // Then update the slides.
    const ID_TITLE_SLIDE = 'id_title_slide';
    const ID_TITLE_SLIDE_TITLE = 'id_title_slide_title';
    const ID_TITLE_SLIDE_BODY = 'id_title_slide_body';

    return [{
        // Creates a "TITLE_AND_BODY" slide with objectId references
        createSlide: {
            objectId: `${ID_TITLE_SLIDE}_${index}`,
            slideLayoutReference: {
                predefinedLayout: 'TITLE_AND_BODY'
            },
            placeholderIdMappings: [{
                layoutPlaceholder: {
                    type: 'TITLE'
                },
                objectId: `${ID_TITLE_SLIDE_TITLE}_${index}`
            }, {
                layoutPlaceholder: {
                    type: 'BODY'
                },
                objectId: `${ID_TITLE_SLIDE_BODY}_${index}`
            }
        ], {
            // Inserts the license name, percent, and count in the title
            insertText: {
                objectId: `${ID_TITLE_SLIDE_TITLE}_${index}`,
                text: `#${index + 1} ${licenseData.licenseName} - ~${licenseData.percent}%`
            }, {
                insertText: {
                    objectId: `${ID_TITLE_SLIDE_BODY}_${index}`,
                    text: `($ ${commaNumber(licenseData.count)} repos)`
                }
            }
        }
    ]
}
```



```

    }
  }, {
    // Inserts the license in the text body paragraph
    insertText: {
      objectId: `${ID_TITLE_SLIDE_BODY}_${index}`,
      text: licenseData.license
    }
  }, {
    // Formats the slide paragraph's font
    updateParagraphStyle: {
      objectId: `${ID_TITLE_SLIDE_BODY}_${index}`,
      fields: '*',
      style: {
        lineSpacing: 10,
        spaceAbove: {magnitude: 0, unit: 'PT'},
        spaceBelow: {magnitude: 0, unit: 'PT'},
      }
    }
  }, {
    // Formats the slide text style
    updateTextStyle: {
      objectId: `${ID_TITLE_SLIDE_BODY}_${index}`,
      style: {
        bold: true,
        italic: true,
        fontSize: {
          magnitude: 10,
          unit: 'PT'
        }
      },
      fields: '*',
    }
  }
}];
}

/**
 * Creates slides for our presentation.
 * @param {authAndGHData} An array with our Auth object and the GitHub data.
 * @return {Promise} A promise to return a new presentation.
 * @see https://developers.google.com/apis-explorer/#p/slides/v1/
 */
module.exports.createSlides = (authAndGHData) => new Promise((resolve, reject) => {
  console.log('creating slides...');
  const [auth, ghData] = authAndGHData;

  // First copy the template slide from drive.
  drive.files.copy({
    auth: auth,
    fileId: '1toV2zL0PrXJOfFJU-NYDKbPx9W0C4I-I8iT85TS0fik',
    fields: 'id,name,webViewLink',
    resource: {
      name: SLIDE_TITLE_TEXT
    }
  });
});

```

```
    }
  }, (err, presentation) => {
    if (err) return reject(err);

    const allSlides = ghData.map((data, index) => createSlideJSON(data, index));
    slideRequests = [].concat.apply([], allSlides); // flatten the slide requests
    slideRequests.push({
      replaceAllText: {
        replaceText: SLIDE_TITLE_TEXT,
        containsText: { text: '{{TITLE}}' }
      }
    });

    // Execute the requests
    slides.presentations.batchUpdate({
      auth: auth,
      presentationId: presentation.id,
      resource: {
        requests: slideRequests
      }
    }, (err, res) => {
      if (err) {
        reject(err);
      } else {
        resolve(presentation);
      }
    });
  });
});
```

步驟 8 · 約 1 分鐘

8. 開啟簡報

最後，請在瀏覽器中開啟簡報。在 `slides.js` 中更新下列方法。

slides.js

```
/**
 * Opens a presentation in a browser.
 * @param {String} presentation The presentation object.
 */
module.exports.openSlidesInBrowser = (presentation) => {
  console.log('Presentation URL:', presentation.webViewLink);
  openurl.open(presentation.webViewLink);
}
```

最後一次執行專案，顯示最終結果。

步驟 9 · 約 0 分鐘

9. 恭喜！

您已成功使用 BigQuery 分析資料，並從中產生 Google 簡報。您的指令碼會使用 Google 簡報 API 和 BigQuery 建立簡報，以報告最常見的軟體授權分析。

可能的改善做法

以下提供一些額外建議，協助您打造更具吸引力的整合功能：

- 在每張投影片中新增圖片
- 使用 Gmail API 透過電子郵件分享投影片
- 以指令列引數的形式自訂範本投影片

瞭解詳情

- 參閱 [Google Slides API 開發人員說明文件](#)。
 - 在 Stack Overflow 使用 [google-slides-api](#) 標籤來發問及尋找答案。
-